SOFTWARE ENGINEERING LAB 5

Sanapala Venkata Maneesh
PES1UG24AM247
AIML 5E

1. Objective

The objective of this lab was to inspect and improve the Python source file **theater_system.py** using static-analysis tools. The work focused on code quality, common security weaknesses, and Python formatting conventions. The program was also run after changes to check that its basic reservation workflow continued to execute.

2. Tools Used

Tool	Purpose
Pylint	Checks code quality, naming, documentation, common programming mistakes, and maintainability.
Bandit	Scans Python code for common security issues and risky patterns.
Flake8	Checks PEP 8 style rules and additional linting issues such as line length and whitespace.

Commit	Commit message	Purpose
5e061c5	Fix Pylint code quality issues	Applied code-quality and maintainability fixes.
39002e7	Fix Bandit security issues	Addressed the reported hashing and file-permission findings.

f71140f	Fix Flake8 formatting issues	Cleaned up style and formatting violations.

3. Issues Identified and Fixes Applied

Tool	Finding	Change made
Pylint	Mutable default argument in reserve_seat()	Changed the default from a shared list to None and initialized a fresh list inside the function.
Pylint	Missing documentation and naming/style issues	Added docstrings and changed printSeatingSummary() to print_seating_summary().
Pylint	Broad exception handling	Replaced bare exception handling with specific exceptions and explicit failure handling.
Pylint	File handling and string formatting	Used context managers, explicit UTF-8 encoding, and f-strings.
Bandit	MD5 hash used for ticket-code generation (B324)	Replaced MD5 with SHA-256 for the ticket identifier.
Bandit	Overly permissive file mode 0o777 (B103)	Removed the chmod call that set broad file permissions.
Flake8	Long lines and spacing violations	Wrapped long statements and corrected blank-line spacing, indentation, and trailing whitespace.

4. Final Results

Tool	Final result	Status
Pylint	10.00 / 10	Target exceeded
Bandit	No issues identified; 0 issues across all reported severities	Passed
Flake8	No output / no reported violations	Passed

5. Git Commit Evidence

The three commits were created separately so the fixes associated with each analysis tool can be reviewed independently.

6. Reflection

Why is static code analysis important?

It helps identify potential defects, security weaknesses, and style violations without requiring the program to execute every possible path. This supports earlier detection and improves maintainability.

What did Pylint help identify?

It highlighted issues such as mutable default arguments, missing documentation, naming/style problems, exception handling, and file-handling practices.

What did Bandit help identify?

It reported the use of MD5 and overly permissive file permissions. The implementation was changed to SHA-256 and the chmod operation was removed.

What did Flake8 help identify?

It identified formatting problems such as long lines, incorrect blank-line spacing, indentation, and trailing whitespace.

How were the fixes verified?

The tools were run again after the edits. Pylint returned 10.00/10, Bandit reported no issues, and Flake8 returned no output. The program was also executed as a basic functional check.

7. Conclusion

The static-analysis exercise improved the readability, maintainability, security posture, and formatting of the theater reservation program. The final source passed all three tools with a Pylint score of 10.00/10 and no reported Bandit or Flake8 issues.

Submission note: Compare the reflection prompts and required evidence against the original lab handout before submitting. If the handout specifies a different report structure or requires screenshots, add those items to the final PDF.